

# Mini Transaction Ledger

## Technical Explanation Report

September 6, 2026

### Contents

|          |                                                |          |
|----------|------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                            | <b>2</b> |
| <b>2</b> | <b>Application Architecture</b>                | <b>2</b> |
| 2.1      | Database schema . . . . .                      | 2        |
| <b>3</b> | <b>Key Code Components and Their Purpose</b>   | <b>3</b> |
| 3.1      | Backend (NestJS) . . . . .                     | 3        |
| 3.2      | Frontend (Next.js / React) . . . . .           | 4        |
| <b>4</b> | <b>API Endpoint Reference</b>                  | <b>4</b> |
| <b>5</b> | <b>How the API Works Internally</b>            | <b>5</b> |
| 5.1      | Request lifecycle . . . . .                    | 5        |
| 5.2      | Authentication and the refresh cycle . . . . . | 6        |
| 5.3      | The ledger's append-only model . . . . .       | 6        |
| <b>6</b> | <b>Docker Setup Explanation</b>                | <b>7</b> |
| <b>7</b> | <b>Conclusion</b>                              | <b>8</b> |

# 1 Introduction

Mini Transaction Ledger is a full-stack application for managing financial accounts and recording transactions against them. A user can create one or more accounts, post credit and debit entries against those accounts, and review a running balance history. An administrator role has a separate view for managing every registered user in the system.

The application is deliberately built around one rule that shapes most of its backend design: **a posted transaction is never edited or deleted**. Corrections are made by posting an offsetting *reversal* entry, so the transaction log always reflects exactly what happened, in the order it happened, permanently.

This document explains how the two halves of the application (frontend and backend) interact, walks through the purpose of the main code components, describes how a request actually flows through the API, and explains the Docker setup used to run the whole stack with one command.

## 2 Application Architecture

The system is split into two independently deployable applications that communicate exclusively over HTTP, plus a shared PostgreSQL database:



The browser (the Next.js client code) talks **directly** to the NestJS API using Axios, with `withCredentials: true` so that authentication cookies are attached automatically. There is no server-side proxy, Route Handler, or backend-for-frontend layer sitting between them — the frontend is a plain single-page-style client application, and the only server the browser ever calls is the NestJS API. Cross-Origin Resource Sharing (CORS) on the backend is restricted to one configured origin (`FRONTEND_URL`) so that only the known frontend can make authenticated, credentialed requests.

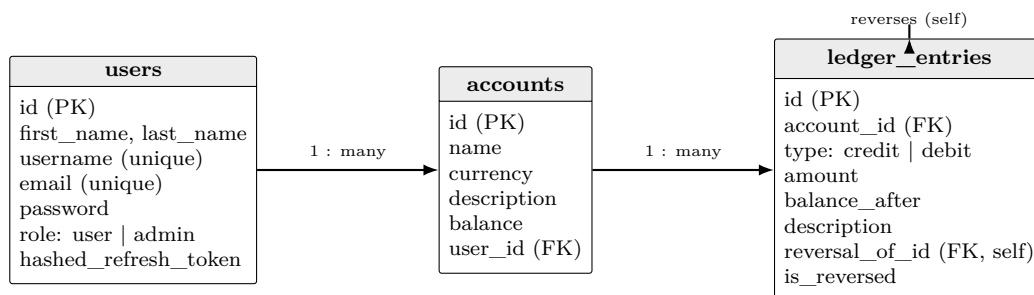
The backend itself is organised into four feature modules, each owning one database entity, its DTOs (Data Transfer Objects), a service (business logic and database access), and a controller (HTTP routes and authorization checks):

- **Auth** — signup, login, token refresh, logout.
- **Users** — the `User` entity, roles, admin-only user management.
- **Accounts** — ledger accounts, one-to-many with `User`.
- **Ledger** — transaction entries, one-to-many with `Account`.

The frontend mirrors this with one dedicated API module per backend resource (`lib/api/auth.ts`, `accounts.ts`, `ledger.ts`, `users.ts`), and a component tree split the same way: an authentication page, and a dashboard that renders one of two completely separate views depending on the logged-in user's role.

### 2.1 Database schema

Three tables model the whole domain: a user owns many accounts, an account owns many ledger entries, and a ledger entry may optionally point back at the entry it reverses.



Two foreign-key behaviours are load-bearing for the "never delete ledger history" rule:

- `accounts.user_id` is `ON DELETE CASCADE` — deleting a user deletes their accounts, *provided* none of them have ledger history.
- `ledger_entries.account_id` is `ON DELETE RESTRICT` — an account (or, by cascade, a user) cannot be deleted while it still has ledger entries. The backend catches this specific database error and turns it into a clean 409 `Conflict` response instead of a raw 500.

## 3 Key Code Components and Their Purpose

### 3.1 Backend (NestJS)

#### `auth/auth.service.ts`

Contains all session logic: hashing passwords with `bcrypt`, issuing an access token (15 minutes) and a refresh token (7 days), and verifying a presented refresh token against a stored hash before issuing a new pair. The refresh token's hash uses `SHA-256` with a constant-time comparison rather than `bcrypt`, because `bcrypt` silently truncates its input to 72 bytes — since every JWT issued to the same user shares an identical header and subject-claim prefix, `bcrypt` would hash them all to the same value, making *any* previously issued token match.

#### `auth/strategies/jwt.strategy.ts` & `refresh-jwt.strategy.ts`

Passport strategies that pull the JWT out of an `httpOnly` cookie (`access_token` or `refresh_token`) rather than an `Authorization` header, and verify its signature and expiry.

#### `auth/guards/jwt-auth.guard.ts`, `roles.guard.ts`

Guards run before a controller method. The first rejects any request without a valid access token; the second reads a `@Roles(...)` decorator's metadata and rejects the request unless the authenticated user's role is in that list.

#### `ledger/ledger.service.ts`

The core business logic for posting and reversing transactions. Both operations run inside a single database transaction with a pessimistic write lock on the account row, so that two concurrent transactions against the same account cannot read a stale balance and compute conflicting results.

#### `accounts/accounts.service.ts`, `users/users.service.ts`

Both `remove()` methods catch the Postgres foreign-key-restriction error codes (23503/23001) and re-throw them as a `ConflictException`, so a doomed delete fails with a clear, actionable message instead of an unhandled database exception.

#### `app.module.ts`

Wires every feature module together and validates all required environment variables at startup using `Joi`, so a missing configuration value fails fast with a clear error instead of

an obscure runtime crash later.

## 3.2 Frontend (Next.js / React)

### `lib/api-client.ts`

A thin wrapper around a single shared Axios instance. Every backend error response is normalized into one `ApiError` class, and a response interceptor transparently retries a failed request once after refreshing the session (see Section 5).

### `lib/api/*.ts`

One file per backend resource, each exporting a small, named function per endpoint (for example `createEntry`, `reverseEntry`, `updateUserRole`). Components never call `fetch/axios` directly — they call these functions, which keeps request shapes and response types centralized and typed.

### `contexts/user-context.tsx`

The application’s single React Context. It fetches the current session once on mount, exposes the logged-in user (or `null`) and a `logout` function, and is the only piece of global state in the whole frontend.

### `components/dashboard/*`

The regular-user dashboard: an account list and a create-account action on the left, and the selected account’s balance, transaction table, and create/edit/reverse actions on the right. State is deliberately “lifted” to the nearest shared ancestor (`UserDashboardView`) rather than duplicated in each child.

### `components/admin/*`

A completely separate dashboard rendered only when the session’s role is `admin`: a table of every user with role-change and delete actions wired directly to the corresponding backend endpoints.

## 4 API Endpoint Reference

Every response is wrapped in the same envelope on success, `{ message, data }`. In the **Access** column, *Admin* means the endpoint requires the `admin` role via `@Roles()/RolesGuard`; *Owner* means it is open to any authenticated user, but the controller additionally checks that the requester either owns the resource or is an admin; *User* means any authenticated user may call it.

### Auth — `/auth`

| Method | Path                       | Description                                                          |
|--------|----------------------------|----------------------------------------------------------------------|
| POST   | <code>/auth/signup</code>  | Register a new user (always created with the <code>user</code> role) |
| POST   | <code>/auth/login</code>   | Log in with email and password                                       |
| POST   | <code>/auth/refresh</code> | Exchange a valid refresh token for a new access/refresh pair         |
| POST   | <code>/auth/logout</code>  | Revoke the current refresh token                                     |
| GET    | <code>/auth/me</code>      | Get the current session’s full user record                           |

## Users — /users

| Method | Path       | Access | Description                                                 |
|--------|------------|--------|-------------------------------------------------------------|
| POST   | /users     | Admin  | Create a user directly                                      |
| GET    | /users     | Admin  | List all users                                              |
| GET    | /users/:id | Owner  | Get a single user                                           |
| PATCH  | /users/:id | Owner  | Update a user (role changes require Admin)                  |
| DELETE | /users/:id | Admin  | Delete a user (fails if their accounts have ledger entries) |

## Accounts — /accounts

| Method | Path          | Access | Description                                                             |
|--------|---------------|--------|-------------------------------------------------------------------------|
| POST   | /accounts     | User   | Create an account for the current user                                  |
| GET    | /accounts     | User   | List accounts (own for <code>user</code> , all for <code>admin</code> ) |
| GET    | /accounts/:id | Owner  | Get a single account                                                    |
| PATCH  | /accounts/:id | Owner  | Update name / description                                               |
| DELETE | /accounts/:id | Owner  | Delete an account (fails if it has ledger entries)                      |

## Ledger entries — /accounts/:accountId/entries

| Method | Path                      | Access | Description                          |
|--------|---------------------------|--------|--------------------------------------|
| POST   | /entries                  | Owner  | Post a credit or debit entry         |
| GET    | /entries                  | Owner  | List entries for the account         |
| GET    | /entries/:entryId         | Owner  | Get a single entry                   |
| PATCH  | /entries/:entryId         | Owner  | Update an entry's description only   |
| POST   | /entries/:entryId/reverse | Owner  | Post a reversal of an existing entry |

## 5 How the API Works Internally

### 5.1 Request lifecycle

A typical protected request (for example, posting a transaction) passes through the following stages inside NestJS:

1. **Guards** — `JwtAuthGuard` verifies the access token cookie and attaches the decoded user (`id`, `email`, `role`) to the request. `RolesGuard` (where applied) checks that role against the endpoint's required roles.
2. **Validation pipe** — the request body is parsed into its DTO class and validated with `class-validator` decorators (`@IsPositive()`, `@IsEnum()`, etc.); unknown properties are stripped, and invalid payloads are rejected with a `400 Bad Request` before any business logic runs.
3. **Controller** — performs any additional ownership check (for example, confirming the account being posted to actually belongs to the requesting user, or that the user is an admin) and delegates to the service.
4. **Service** — contains the actual business logic, wrapped in a database transaction where atomicity matters (posting or reversing a ledger entry).
5. **Serialization** — the response entity passes through a global `ClassSerializerInterceptor`, which strips fields marked `@Exclude()` (the password hash, the refresh token hash) before the JSON reaches the client.

Every successful response is wrapped in the same shape, `{ message, data }`, which the frontend's `ApiError` handling and per-endpoint functions rely on consistently.

## 5.2 Authentication and the refresh cycle

Two JSON Web Tokens are used, both delivered as `httpOnly` cookies so they are never reachable from client-side JavaScript:

- An **access token** (15 minutes), carrying the user's id, email, and role, required on every protected request.
- A **refresh token** (7 days), used only to obtain a new access token. Its hash is stored on the user record so that a stolen or replayed refresh token can be detected: if a presented token does not match the stored hash, the stored hash is cleared, immediately revoking that session.

Because the access token's role claim is only as fresh as the moment it was issued, a role change made through the admin dashboard does not take effect for that user's already-open session until their token is refreshed. Rather than requiring a manual logout, the frontend handles this automatically:

Listing 1: Automatic refresh-and-retry, from `lib/api-client.ts`

```
1 httpClient.interceptors.response.use(  
2   (response) => response,  
3   async (error) => {  
4     const config = error.config;  
5     const status = error.response?.status;  
6  
7     const canRetry =  
8       config &&  
9       !config._retriedAfterRefresh &&  
10      (status === 401 || status === 403) &&  
11      !AUTH_RETRY_EXEMPT_PATHS.some((p) => config.url?.startsWith(p)  
12      );  
13  
14     if (canRetry) {  
15       config._retriedAfterRefresh = true;  
16       try {  
17         await refreshSession(); // POST /auth/refresh  
18         return await httpClient.request(config);  
19       } catch {  
20         // refresh failed too; fall through to the original error  
21       }  
22     }  
23     return Promise.reject(error);  
24   },  
25 );
```

Any 401/403 response (other than from the auth endpoints themselves) triggers exactly one call to `POST /auth/refresh`, which re-reads the user from the database and re-issues both tokens with the current role; the original request is then retried once with the fresh cookies already in place. Concurrent failures share a single in-flight refresh call rather than each starting their own.

## 5.3 The ledger's append-only model

Posting a transaction never simply increments or decrements a balance in isolation — it is always done inside one database transaction that:

1. Acquires a pessimistic write lock on the account row.
2. Computes the new balance from the currently locked value.

3. Inserts the new `ledger_entries` row, stamped with that resulting `balance_after`.
4. Writes the new balance back onto the account row.

Listing 2: Simplified core of `LedgerService.createEntry`

```
1 return this.dataSource.transaction(async (manager) => {
2   const account = await manager.findOne(Account, {
3     where: { id: accountId },
4     lock: { mode: 'pessimistic_write' },
5   });
6
7   const balanceAfter = this.applyEntry(account.balance, dto.type,
8     dto.amount);
9
10  const entry = manager.create(LedgerEntry, {
11    accountId, type: dto.type, amount: dto.amount,
12    balanceAfter, description: dto.description ?? null,
13  });
14
15  account.balance = balanceAfter;
16  await manager.save(account);
17  return manager.save(entry);
18 });
```

Because ledger entries model real financial events, they are never updated or deleted after posting — only the `description` field can be edited afterwards, via a separate endpoint. A mistake is corrected by *reversing* an entry: a new entry of the opposite type, for the same amount, is posted and linked back to the original via `reversal_of_id`, while the original is flagged `is_reversed = true`. This keeps every entry that was ever posted permanently visible, mistakes included, which is the entire point of an audit-grade ledger.

## 6 Docker Setup Explanation

The whole stack — database, API, and frontend — is brought up with a single `docker compose up -build` from the repository root. The root `docker-compose.yml` defines three services:

### postgres

Uses the official `postgres:16-alpine` image directly (no custom Dockerfile needed). A named volume (`postgres_data`) persists data across container restarts, and a `healthcheck` runs `pg_isready` so that the backend does not attempt to connect before the database is actually ready to accept connections.

### backend

Built from `backend/Dockerfile`, a three-stage build: a `deps` stage installs dependencies with `pnpm` (including the `python3/make/g++` toolchain that `bcrypt`'s native module needs to compile on Alpine Linux), a `build` stage runs `nest build` to produce `dist/`, and a final `runtime` stage copies only the compiled output and `node_modules` into a fresh image and runs `node dist/main`. Configuration (database credentials, JWT secrets, the frontend's origin for CORS) is injected entirely through environment variables at container start, sourced from the root `.env` file via Compose's variable substitution — `DB_HOST` is overridden to the service name `postgres` so the backend reaches the database over the Compose-managed Docker network rather than `localhost`.

### frontend

Built from `frontend/Dockerfile`, following the same three-stage pattern, but using Next.js's output: `"standalone"` build mode, which produces a minimal, self-contained

server bundle. Because `NEXT_PUBLIC_BACKEND_URL` is inlined into the client JavaScript bundle at build time (not read at runtime), it is passed in as a Docker build `ARG` rather than a plain environment variable, and is set to `http://localhost:3000` — the address the *browser*, not the container, needs to reach the API on the published host port.

Only the database port (5432), the API port (3000), and the frontend port (3001) are published to the host, matching exactly how the same three services are addressed when run locally without Docker. A single root `.env.example` (copied to `.env` before the first run) supplies every value the compose file's variable substitution needs, so a fresh clone requires no manual configuration beyond that one copy.

## 7 Conclusion

The application separates concerns cleanly along two axes: a frontend/ backend split communicating over a typed, credentialed REST API, and a backend split into four self-contained feature modules. Authentication uses short-lived, role-bearing access tokens backed by a longer-lived, rotation-aware refresh token, with the frontend transparently keeping itself in sync when a role changes mid-session. The ledger itself is modelled as an append-only log with a self-referencing reversal pattern and database-level delete protection, so that the system's core guarantee — transaction history is permanent — is enforced at the database, not just trusted to application code. The entire stack, including the database, is reproducible with a single Docker Compose command.